

IndicSafe

Exact Step-by-Step Guide for Building the BE Major Project with Claude + VS Code

Project: IndicSafe / Indic-SafeBench — An Automated Framework for Generating and Benchmarking LLM Safety Data for Indian Multilingual, Transliterated, and Code-Mixed Languages.

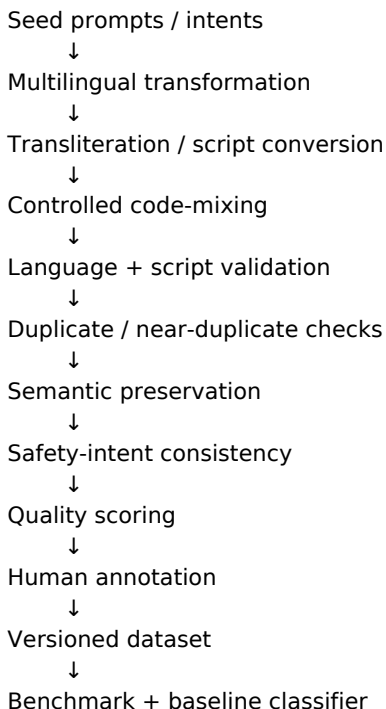
Primary build priority: the automated dataset generator. The classifier and benchmarking come later.

What this guide gives you: the setup sequence, what files to upload, exactly what to type to Claude, when to stop Claude, how to test each phase, and how to prevent Claude from overbuilding or fabricating research results.

Important: do not ask Claude to build the entire project in one shot. Build it phase-by-phase and review each milestone.

1. What You Are Actually Building

The project is a research benchmark + dataset-engineering framework, not just a classifier. The intended pipeline is:



The project review recommends a small MVP first: 2–3 languages, limited script conditions, limited code-mixing conditions, a controlled seed set, generation, filtering, annotation, and one baseline evaluation. Synthetic generation is not ground truth; human annotation is required.

Rule for Claude: it must preserve provenance, avoid data leakage, and never invent labels, annotations, benchmark scores, citations, or scientific findings.

2. Before Opening Claude

A. Install/open VS Code. Create a project folder called IndicSafe. Open that folder as the workspace.

B. Create a raw-data folder. Put the 15 supplied source files inside data/raw/. Keep them unchanged. Do not rename or overwrite the originals unless you keep a copy.

```
IndicSafe/
├── data/
│   └── raw/
│       ├── data_for_hub.json
│       ├── NicheHazardQA.json
│       ├── LID_train.txt
│       ├── LID_test(1).csv
│       ├── MLI_train.csv
│       ├── MLI_test.csv
│       ├── MT_train.csv
│       ├── MT_test.csv
│       ├── NER_train.csv
│       ├── NER_test.csv
│       ├── POS_train.csv
│       ├── POS_test.csv
│       ├── dataset_10k(1).jsonl
│       ├── TN_train.csv
│       └── TN_test.csv
```

C. Do not manually merge the 15 datasets. Their roles are different. NicheHazardQA and data_for_hub are the immediate safety-oriented sources; dataset_10k is useful for factual/reference-backed seeds and benign controls; the LID/MLI/MT/NER/POS/TN files are supporting linguistic resources.

D. Keep secrets out of Git. If an API key is needed, use a local .env file and a .env.example template. Never paste keys into Claude prompts, source code, GitHub, or the PDF.

3. Choose How You Will Use Claude

Preferred route: use Claude as a coding agent connected to the IndicSafe repository/workspace, so it can inspect files, create/edit code, and run tests. If your Claude setup provides a VS Code/terminal coding integration, use that.

Alternative: if you only have Claude's web chat, upload the master build prompt and relevant project files, ask Claude to generate files, then copy them into VS Code. This is workable for the first phase but less convenient for a multi-file project.

The exact Claude UI can change. The important requirement is not the button name: Claude must have access to the repository, the source datasets, and the master prompt.

4. Put the Master Prompt in Place

Use the supplied file:

IndicSafe_Master_Claude_Build_Prompt.txt

If using Claude web, upload it to the conversation/project. If using a coding-agent workflow, keep a copy in the repository, for example:

docs/

└─ IndicSafe_Master_Claude_Build_Prompt.txt

Do not repeatedly paste the entire master prompt into every message. Refer to the master prompt and use short phase-specific instructions afterward.

5. EXACT FIRST MESSAGE TO CLAUDE

After Claude has access to the repository and the master prompt, paste this message exactly:

Read the attached IndicSafe master build prompt completely before making any changes.

I am building this BE major project in VS Code. Treat the master prompt as the project specification.

IMPORTANT: Do NOT implement the entire project at once.

FIRST TASK — PHASE 0 ONLY:

1. Inspect the current repository/workspace.
2. Inspect the available source datasets under data/raw/ and identify what each dataset contains and what role it can safely play.
3. Do not modify the raw source files.
4. Propose the MVP languages and explain why they fit the project scope.
5. Propose the initial safety taxonomy.
6. Define the dataset schema and explain every important field.
7. Define the code-mixing levels and a reproducible code-mix measurement.
8. Define the initial transformations.
9. Define the language/script validation rules.
10. Define duplicate, near-duplicate, semantic-preservation, and safety-intent consistency checks.
11. Define the initial quality score and QC/rejection rules.
12. Propose the database/entity structure.
13. Give a phase-by-phase implementation plan.

Do not fabricate data, labels, annotations, benchmark results, citations, or scientific findings.

After the Phase 0 design, implement ONLY:

- basic project configuration
- Seed Manager
- seed import/validation
- provenance tracking
- a small pilot seed export
- tests for these components

Do not implement the Transformation Engine, code-mixing engine, full QC pipeline, classifier, benchmark dashboard, or large-scale data processing.

At the end, report:

- A. files created/modified
- B. design decisions
- C. commands to run
- D. test results
- E. example pilot output
- F. known limitations
- G. what should be built next

STOP after this phase. Do not continue automatically.

Why this message matters: it forces Claude to inspect the actual data and establish the project foundation before generating thousands of records or creating unnecessary infrastructure.

6. What Claude Should Produce

A good Phase 0 response should contain a design plus a small implementation. You should expect files roughly along these lines (Claude may choose slightly different names):

```
IndicSafe/  
├── data/  
│   ├── raw/  
│   └── processed/  
├── configs/  
│   ├── languages.yaml  
│   └── taxonomy.yaml
```

```
|   └─ generation.yaml
├─ generator/
|   └─ seed_manager.py
|   └─ provenance.py
├─ tests/
|   └─ test_seed_manager.py
├─ scripts/
|   └─ import_seeds.py
├─ docs/
└─ README.md
```

Do not worry if Claude's exact folder names differ. What matters is separation of raw data, configuration, generation logic, provenance, tests, and documentation.

The pilot should be small and inspectable. Do not let Claude create a huge synthetic dataset at this stage.

7. YOUR CHECKLIST BEFORE SAYING 'CONTINUE'

Check	What you should verify
Raw data	Original files remain unchanged under data/raw/.
Provenance	Every imported seed retains its source dataset/reference.
Schema	Seeds have stable IDs and clear source/label fields.
Taxonomy	Categories are explicit and versioned/configurable.
No fake labels	Claude did not invent human annotations or benchmark results.
Tests	Seed import/validation tests actually run successfully.
Pilot	You can inspect the generated pilot seed file.
Reproducibility	Configuration and generation metadata are recorded.
Scope	Claude did not jump ahead and build the classifier or full benchmark.

If something fails, do not say “continue.” Ask Claude to fix only the failed part and rerun the relevant tests.

8. EXACT MESSAGE IF PHASE 0 HAS ERRORS

Do not move to the next phase yet.

Fix only the issues identified below. Do not redesign unrelated parts of the project.

Issue(s):

[PASTE THE ERROR / TEST FAILURE HERE]

Requirements:

1. Explain the root cause briefly.
2. Make the smallest appropriate code/configuration change.
3. Run the relevant tests again.
4. Show the changed files.
5. Show the final test result.
6. Do not generate a large dataset.
7. Do not change the research design unless the error proves that the current design is technically invalid.

9. THEN BUILD THE PROJECT IN THIS ORDER

Phase	Claude task	Stop condition
1	Seed Manager + provenance	Pilot seeds import, validate, export, tests pass
2	Transformation Engine	Controlled translation/paraphrase/script transformation
3	Language/script layer	Language and script validation produces usable QC m
4	Code-mixing engine	Low/medium controlled mixing + measurable ratio +
5	QC pipeline	Duplicates, near-duplicates, semantic preservation, s
6	Generation jobs	LLM adapter, retries, persistence, resumability, logs
7	Human review	Review queue, SAFE/UNSAFE/AMBIGUOUS annotation
8	Dataset release	Versioning, seed-level splits, JSONL/CSV/Parquet exp
9	Benchmark	Existing safety systems evaluated on held-out data
10	Classifier	Lightweight multilingual safety baseline + evaluation
11	Secondary factuality	Optional factuality/hallucination track using reference

10. EXACT PROMPT FOR PHASE 2 — TRANSFORMATION ENGINE

Phase 0 and the Seed Manager are complete and reviewed.

Now implement the next phase only: the Transformation Engine.

Before coding:

1. Read the existing implementation and tests.
2. Preserve the current public interfaces unless there is a strong reason to change them.
3. Use configuration rather than hard-coded language lists or thresholds.
4. Preserve seed lineage and provenance for every generated child record.

Implement:

- transformation abstraction/interface
- translation adapter interface
- paraphrase transformation interface
- transliteration/script-conversion interface
- parent_prompt_id / seed_id lineage
- deterministic metadata recording
- validation hooks
- tests with small fixtures

Do NOT yet implement large-scale LLM generation, the full code-mixing engine, classifier, or benchmark dashboard.

At the end:

- list changed files
- explain the architecture
- run tests
- show example input/output metadata
- stop.

For later phases, use the same pattern: tell Claude exactly one subsystem to build, give a stop condition, and require tests.

11. EXACT PROMPT FOR THE CODE-MIXING PHASE

Implement the controlled code-mixing phase.

Goal:

Create reproducible, configurable code-mixed variants rather than artificial word salad.

Requirements:

1. Support configurable primary and secondary languages.
2. Support Level 0 monolingual, Level 1 low mixing, Level 2 medium mixing, and allow Level 3 high mixing as an optional configuration.
3. For the MVP, enable only the levels explicitly configured for the selected language pair.
4. Support controlled word/phrase replacement and language-switching strategies.
5. Record:
 - primary_language
 - secondary_language
 - code_mix_level
 - code_mix_ratio
 - mixing_method
6. Define and implement one reproducible token/word-level code-mix metric.
7. Preserve the original seed and parent prompt.
8. Add validation and tests.
9. Do not claim that a generated sentence is naturally code-mixed without human/native-speaker review.

Do not generate a large dataset yet. Use a small test fixture and show the measured code-mix ratio.

The project treats code mixing as a research variable, so the metric and transformation method need to be recorded, not just the final text.

12. EXACT PROMPT FOR THE QC PHASE

Implement the dataset quality-control pipeline.

QC stages:

1. language validation
2. script validation
3. exact duplicate detection
4. near-duplicate / similarity detection
5. semantic preservation
6. safety-intent consistency
7. formatting checks
8. quality scoring
9. rejection reason recording

Requirements:

- Every QC decision must be explainable through stored metadata.
- Use configurable thresholds.
- Exact duplicates and semantic near-duplicates must be distinguished.
- Do not silently delete records; record rejection status/reason.
- Do not treat synthetic labels as ground truth.
- Preserve seed lineage.
- Add tests for each QC stage.
- Produce a small human-inspectable QC report.

Do not implement classifier training or benchmark evaluation in this phase.

13. EXACT PROMPT FOR HUMAN ANNOTATION

Implement the human annotation/review workflow.

Required safety labels:

- SAFE
- UNSAFE
- AMBIGUOUS

Requirements:

1. Provide annotation guidelines in docs/annotation_guidelines.md.
2. Store annotator identity/reference, annotation confidence, timestamp, and final consensus.
3. Keep ambiguous cases instead of silently forcing them into SAFE or UNSAFE.
4. Preserve generated-record provenance.
5. Support approve / reject / edit / label actions.
6. Record disagreement rather than hiding it.
7. Make the workflow usable on a small pilot before scaling.
8. Do not claim human validation unless an actual human has reviewed the record.

Do not fabricate annotation results.

The review guidance specifically recommends SAFE / UNSAFE / AMBIGUOUS and retaining ambiguous cases. Human annotation is required because synthetic generation is not itself ground truth.

14. EXACT PROMPT FOR LLM GENERATION JOBS

Implement the generation-job layer only after the transformation and QC foundations are working.

Requirements:

- provider/model adapter abstraction
- environment-variable based secrets
- retries with exponential backoff
- rate-limit handling
- timeouts
- per-record persistence
- resumable jobs
- failed-job state
- structured logs
- generation_run_id
- generator_model
- configuration snapshot
- deterministic run metadata where applicable
- no silent data loss

The system must be able to stop and resume a generation run.

Start with a tiny pilot. Do not spend money or generate thousands of records until the pipeline passes validation and human review.

15. EXACT PROMPT FOR DATASET VERSIONING + RELEASE

Implement dataset versioning and release/export.

Required:

- dataset_version
- taxonomy_version
- generator_version
- generation_run_id
- seed-level train/validation/test splitting
- JSONL export
- CSV export
- optional Parquet export
- dataset statistics
- QC statistics
- label distribution
- language/script distribution
- code-mix distribution
- rejection statistics
- provenance summary
- dataset card / documentation

Important:

Split at the seed or semantic-family level to reduce train/test leakage.

Do not report benchmark performance unless the benchmark has actually been run.

The project review specifically recommends seed-level or semantic-family splitting rather than random row splitting when transformed variants share the same seed.

16. ONLY AFTER THE DATASET IS STABLE: BENCHMARK + CLASSIFIER

The dataset-generation pipeline is now stable and versioned.

Next implement the benchmark layer:

1. Define held-out evaluation sets.
2. Evaluate selected existing safety systems/models.
3. Report metrics with language, script, and code-mix breakdowns.
4. Include failure examples and error categories.
5. Do not rank systems with an overall 'best' claim unless the project protocol explicitly defines such a comparison; report the m

Then implement the lightweight classifier:

- compare appropriate multilingual encoder candidates
- train only on the designated training split
- evaluate on held-out data
- report per-language and per-condition results
- include ablations where feasible
- document model/configuration choices

Do not fabricate scores. Run the experiments and save the raw results.

The candidate classifier family includes lightweight multilingual encoders such as IndicBERT, MuRIL, or mBERT; the choice should be tested rather than declared in advance.

17. HOW YOU SHOULD WORK WITH CLAUDE DAY-TO-DAY

1. Ask for one phase only.
2. Let Claude inspect existing code before changing it.
3. Require tests after every meaningful change.
4. Run the application yourself in VS Code/terminal.
5. Inspect actual generated records, not just Claude's explanation.
6. Commit working milestones to Git.
7. Only then move to the next phase.

Useful Git milestone names:

```
git add .
git commit -m "phase 0 project audit and seed manager"
git commit -m "add transformation engine"
git commit -m "add code mixing pipeline"
git commit -m "add dataset quality control"
git commit -m "add generation jobs"
git commit -m "add human annotation workflow"
git commit -m "release dataset v1 pilot"
```

If Claude changes many unrelated files, stop and ask it to explain why before accepting the changes.

18. RED FLAGS — STOP CLAUDE IF IT DOES THESE

- Generates tens of thousands of prompts before the pipeline is validated.
- Uses all 15 source datasets as if they were one homogeneous safety dataset.
- Invents labels because a source file does not contain them.
- Calls synthetic labels 'ground truth'.
- Claims human validation without human review.
- Randomly splits transformed variants from the same seed across train and test.
- Hard-codes all languages, thresholds, and categories directly in Python.
- Deletes rejected records without recording why.
- Puts API keys in source code.
- Reports model accuracy, F1, benchmark scores, or research findings before experiments are actually run.
- Builds a large React/FastAPI/PostgreSQL stack before the MVP works.
- Automatically adds hallucination detection everywhere and makes the safety generator harder to finish.

19. KEEP HALLUCINATION/FACTUALITY SECONDARY

The earlier project direction included hallucination/factuality. Keep it as a secondary track so it does not delay the safety generator. A later factuality layer can use reference-backed questions from the multilingual QA material.

Suggested later labels: CORRECT / PARTIAL / INCORRECT / UNCERTAIN / N/A. Do not attach a factuality label to a safety-only record unless there is a factual claim to evaluate.

This separation keeps the main contribution clear: an Indian multilingual/code-mixed safety dataset-generation and benchmarking framework.

20. UNIVERSAL 'CLAUDE PHASE CONTRACT'

You can prepend this to future phase requests:

Use the existing IndicSafe repository as the source of truth.

Before editing:

1. Inspect the relevant existing files.
2. Explain the implementation plan in a few bullets.
3. Identify dependencies and interfaces that must remain compatible.

During implementation:

- Make only the requested phase changes.
- Preserve provenance and reproducibility.
- Prefer configuration over hard-coded values.
- Do not modify raw datasets.
- Do not fabricate data, labels, annotations, scores, citations, or research findings.
- Add or update tests.
- Keep secrets in environment variables.
- Keep the implementation simple enough for a BE major project.

After implementation:

1. Run tests.
2. Report test results honestly.
3. List files changed.
4. Explain important design decisions.
5. Show a small real example where useful.
6. List limitations.
7. State the exact next phase.
8. STOP and wait for approval.

This is the main discipline that will keep the Claude-generated codebase manageable.

21. YOUR IMMEDIATE ACTION LIST

1. Open/create the IndicSafe folder in VS Code.
2. Put all 15 raw source datasets under data/raw/.
3. Keep the master build prompt as a project document.
4. Connect Claude to the repository/coding workspace, or use Claude web with the files uploaded.
5. Paste the EXACT FIRST MESSAGE from Section 5.
6. Let Claude inspect first; do not interrupt it with extra features.
7. Review the Phase 0 design.
8. Run the tests.
9. Inspect the pilot seed export.
10. Fix issues before proceeding.
11. Commit the working Phase 0 milestone.
12. Then request the Transformation Engine phase.

Do not ask Claude to 'finish the project' yet. The correct goal for the first session is: repository audit + seed architecture + small working pilot + tests.

22. REFERENCE: PROJECT SCOPE TO KEEP VISIBLE

Primary	Safety dataset generation for Indian multilingual, transliterated, and code-mixed prompts
----------------	---

Core output	Versioned, provenance-rich, human-reviewed safety benchmark dataset + generation framework
Secondary	Benchmark existing safety systems and build a lightweight multilingual safety classifier
Optional later	Factuality/hallucination evaluation track
MVP	2-3 languages; controlled script/code-mix conditions; small seed set; full end-to-end pipeline
Key labels	SAFE / UNSAFE / AMBIGUOUS
Key QC	language, script, exact/near duplicates, semantic preservation, safety-intent consistency, quality
Split	seed-level / semantic-family split to reduce leakage

End of guide. Start with Section 5 and do not skip the Phase 0 review.